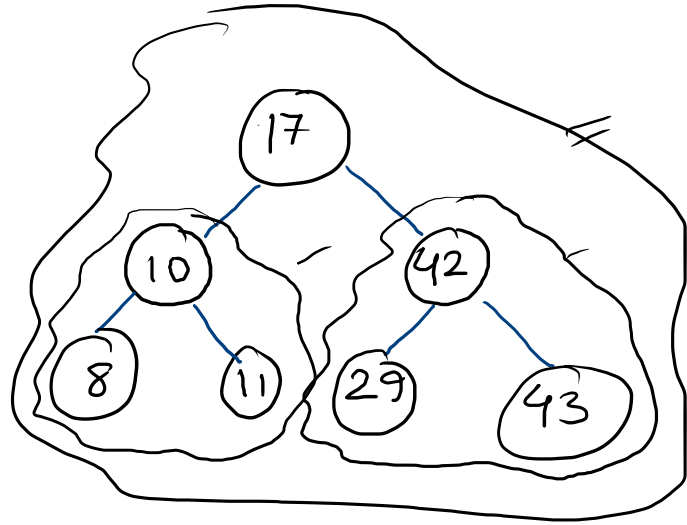
Chapter – 11.2.

Application of trees.



(*) Binary Search Tree: (BST)

- value child < parent → left node
- value child > parent → right node.

Construct BST with 17, 10, 42, 11, 8, 29, 43

(*) Decision tree → m-ary tree.

- rooted tree
- internal vertices represent a decision
-

⊛ Prefix codes.

- Encode 26 letters of EA using bit string
- $2^5 = 32 > 26$ → |bitstring| = 5
- Can we encode efficiently < less memory ? / less Tx time ?

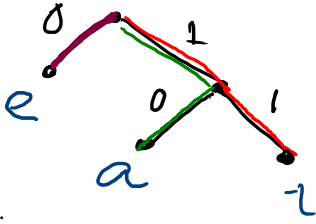
⊛ Consider encoding using varying length bit-strings.

- Start and end
- represent frequent letters with shorter bit strings.

- e → 0, a → 1, t → 01 → 0101 → tt, eat, tea, eaea → ambiguity.
- to remove ambiguity → bit string for a letter never occurs as the first part of another letter. → Prefix codes

- e → 0, a → 10, t → 11 → 0101 → eat

- Encode using rooted binary tree
  - 0 → left edge
  - 1 → right edge
  - chars → leaf nodes.

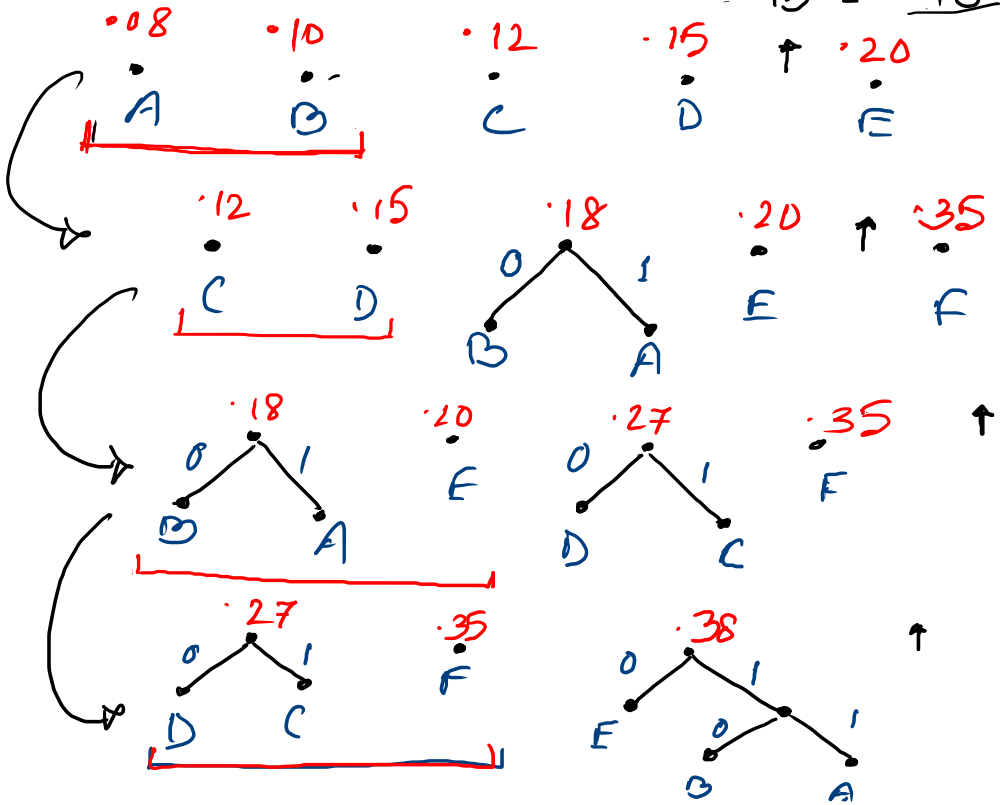


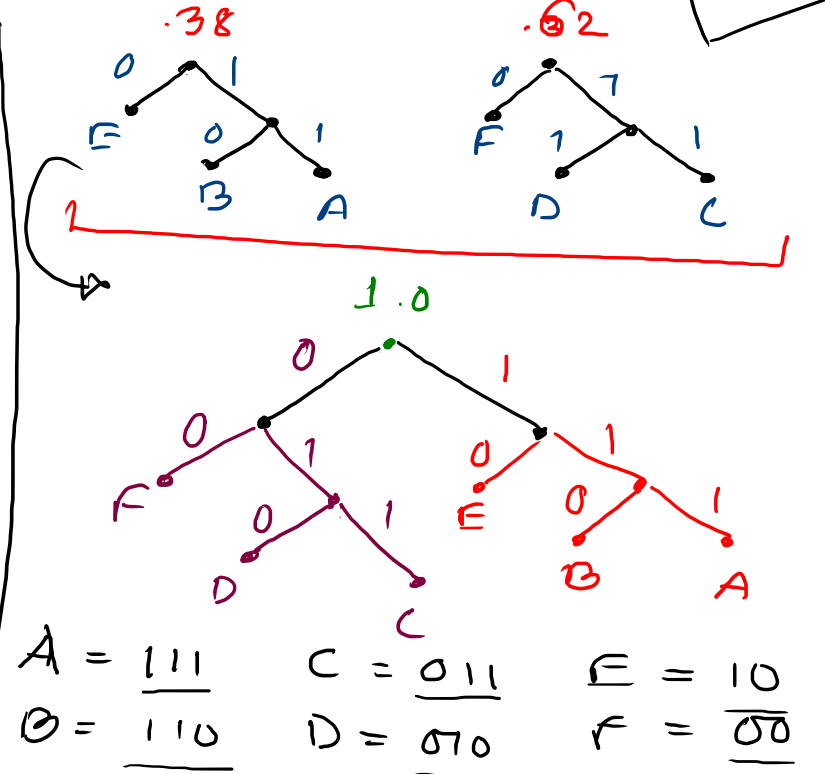
⑦ Huffman coding

- INPUT: Frequency / Probabilities of symbols in a string
- OUTPUT: A prefix code with the fewest possible bits
- GOAL: Given the frequencies, generate the rooted binary tree.

- Generate HC for A = .08, B = .10, C = .12, D = .15, E = .20, F = .35

2.45





✳ Game trees.

- Vertices → current state of game
- Edges → legal moves.
- Leaves → final outcome of the game.
- root → Starting point.
- { vertex in even level → 1st player
  " " odd " → 2nd player

✳ Game of Nim

✳ Tic-Tac-Toe